

Number: P3933R0

Date: 2025-12-14

Audience: [Library Evolution](#)

Target: C++26

Author: [Hana Dusíková](#)

-
- [Motivation](#)
 - [History of `std::hive` and `constexpr`](#)
 - [Implementation](#)
 - [get_iterator and comparing of unrelated pointers](#)
 - [to get_iterator or not to get_iterator now](#)
 - [Wording](#)
 - [Feature test macro](#)

Thanks to NylteJ for providing implementation in MS STL.

P3933R0: `constexpr` hive

This paper makes `std::hive` usable during constant evaluation, resolving NB comment [CZ 1-231 23.3.8 \[hive\] make `std::hive constexpr`](#) and making everything in section `containers` consistently marked `constexpr`.

Motivation

C++ users are often surprised by missing functionality available during constant evaluation, sometimes there is no real reason in the language, it's just no one wrote a paper making the thing `constexpr`. Anecdotally this is often surprise to Jason Turner's students when they start experimenting with `constexpr` code.

This paper rectify this, and makes section [\[containers\]](#) fully `constexpr` compatible. This is also design intent of [P3372: `constexpr` containers and adapters](#) which made all containers marked `constexpr`, `std::hive` which got merged after this paper is not marked `constexpr`.

We believe it's important to have it `constexpr` now otherwise a valid implementation strategy would make future addition of `constexpr` ABI break (by forcing specific layout).

History of `std::hive` and `constexpr`

- **2017-10-16:** The paper [P0447R4](#) introducing `std::hive` under original "colony" name was first published.
- **2019-08-23:** `constexpr` [std::string](#) and [std::vector](#) containers are adopted in C++20 draft just before feature freeze (huge thanks for the paper to Louis Dionne). This doesn't contain non-transient allocations, no memory allocation done in compile time can be transferred to runtime.

- 2020-03-24: Clang 10 with constexpr non-transient allocations support was released.
- 2020-05-07: GCC 10.1 with constexpr non-transient allocations support was released.
- 2021-02-16: LEWG [asked the author to consider](#) making the `std::hive` constexpr.
- 2021-03-25: [The paper in its 13th revision](#) introduces appendix "why not constexpr?". There the author [describes their worries](#) about lowering performance of the container by compiler deciding to replace calls like `size()` with a constant somewhere in memory, instead of doing effective calculation in cache. The author claims benchmarked constexpr implementation resulted in 2% slowdown which according them is unacceptable.

Note this is not how the constant evaluation is specified, only constant-known values can be used in constant evaluation, so in order for constant evaluator to replace `size()` with a value in runtime it would need to have both iterators as constant-known values. Such replacements are by optimizer, not the constant evaluator.

[Another worry of the author is bloated binary size when resources are precalculated.](#) If you don't want to have precalculated resources in your binaries, then do not precalculate them. Compilers constant evaluates code only when requested thru constexpr variables initialization, if `constexpr`, template arguments, and similar constructs. This is not happening for random constexpr code, and again this happens in most cases with optimizers taking advantage of code visibility.

["If there were a mechanism which specified that for a given class instance, it's constexpr functions may not be evaluated at compile time, then I would give the go-ahead."](#) this is how constant evaluated code is behaving, any constant folding and inlining is done by optimizer regarding constexpr-ness.

- 2021-11-03: in [R17](#) of the paper the appendix "why not constexpr?" is simplified by removing the list of claims about constexpr functions.
- 2022-02-18: in [R19](#) of the paper authors proposes wait and see approach to making `std::hive` marked constexpr.
- 2022-06-04: in [R20](#) The author says adding of constexpr can be done later before ABI freezes.

Note: this paper argues that change should be done now as part of C++26, so we won't need to break ABI.

- 2023-10-09: in [R23](#) authors updates the appendix again and says: *["...Early compiler support was not good but this is improving steadily over time. I wasn't happy with having to label each and every function as constexpr, as this seemed to prompt some compilers to store some results at compile time even when the container wasn't being used as constexpr, bloating executable size and cache use..."](#)*, the author also argues necessity of usage of `reinterpret_cast` in various parts of the data structure to provide optimal code-gen and layout.

Note: the reference implementation is using `reinterpret_cast` between two unrelated structs (which both contains array of bytes) and then casting them to `element_type` or the skiplist element type. Such `reinterpret_cast`-ing is a language undefined behaviour. Closest casting which is valid is to/from same type and byte storage, which is not allowed in `constexpr` yet, but can be replaced with usage of union.

Author also states *"I am personally happier for `std::array` and `std::vector` to be the 'canaries in the coalmine' here"*.

- **2023-11-10:** `std::hive` was forwarded by LEWG to LWG pending electronic polling (12/5/3/1/0).
- **2023-11-10:** `std::hive` was forwarded with weak consensus to LWG in [electronic polling](#) (8/4/3/4/2).
- **2024-10-08:** [P3372R0](#) "constexpr containers and adapters" (making *all* containers in the draft `constexpr`) got its design approved by LEWG (10/2/2/0/0) while requesting some fixes of problems found (getting non-const reference to const key of map's node, a language UB).
- **2025-01-28:** [P3372R2](#) "constexpr containers and adapters" was forwarded to LWG (12/8/2/0/0).
- **2025-02-15:** [P3372R3](#) and [P0447R28](#) (`std::hive`) was adopted to C++26 during Hagenberg meeting.

Implementation

[NylteJ](#) provided implementation of `constexpr std::hive` in his [fork of MS STL](#), and informed me [under the issue](#) for `std::hive` implementation tracking in MS STL where Steven T. Lavavej made a note that whoever implements it, should make it sure it can be `constexpr` compatible. According the author only problematic thing is `::get_iterator(const T*)` functionality.

`get_iterator` and comparing of unrelated pointers

Wording requirement for `get_iterator` is to be linear with number of blocks allocated. During constant evaluation there is no global ordering of pointers (even in runtime it's just implementation defined behaviour). And implementation can't blindly compare pointers to find in which block and calculate offset to get the iterator. So for now this paper doesn't propose making this function marked `constexpr`.

Alternative would be ignore complexity requirement during constant evaluation and do a linear scan. But that's really suboptimal. For a future language and library development we need to get a functionality to check if two pointers are associated with same object. Then we can calculate differences of this pointer and start of the block, so we can calculate index of the item for our iterator.

to `get_iterator` or not to `get_iterator` now

I wrote a proposal [P3852 \(alternative link here\)](#) to create a query mechanism to check if a pointer belongs to a memory area between two pointers. This needs a compiler magic builtin. Basically we have two options, add `constexpr` now to `get_iterator` or wait for C++29 with proposed functionality and add it later. The check is implemented this way:

```
constexpr bool is_pointer_between(const void * ptr, const void * first, const void * last) {
    if constexpr {
        if (!__builtin_pointer_related(first, ptr) || !__builtin_pointer_related(ptr, last)) {
            return false;
        }
    }
    return first <= ptr && ptr < last;
}
```

This or a similar builtin mechanism can land independently in compilers and it wouldn't be first time a library functionality needed some magic.

Wording

Wording adds `constexpr` to all functions and member functions in [\[hive.syn\]](#) and [\[hive\]](#). Following diff is made against version of C++26's draft from November 8th, so it probably won't contain changes applied after the Kona meeting, if there is a conflict, intention is to just add `constexpr` everywhere.

23.3.8 Header <hive> synopsis

[hive.syn]

```
#include <initializer_list>           // see [initializer.list.syn]
#include <compare>                     // see [compare.syn]

namespace std {
    struct hive_limits {
        size_t min;
        size_t max;
        constexpr hive_limits(size_t minimum, size_t maximum) noexcept
            : min(minimum), max(maximum) {}
    };

    // [hive], class template hive
    template<class T, class Allocator = allocator<T>> class hive;

    template<class T, class Allocator>
```

```
constexpr void swap(hive<T, Allocator>& x, hive<T, Allocator>& y)
    noexcept(noexcept(x.swap(y)));

template<class T, class Allocator, class U = T>
constexpr typename hive<T, Allocator>::size_type
    erase(hive<T, Allocator>& c, const U& value);

template<class T, class Allocator, class Predicate>
constexpr typename hive<T, Allocator>::size_type
    erase_if(hive<T, Allocator>& c, Predicate pred);

namespace pmr {
    template<class T>
        using hive = std::hive<T, polymorphic_allocator<T>>;
}
}
```

23.3.9 Class template `hive`

[hive]

23.3.9.1 Overview

[hive.overview]

- 1 A `hive` is a type of sequence container that provides constant-time insertion and erasure operations. Storage is automatically managed in multiple memory blocks, referred to as *element blocks*. Insertion position is determined by the container, and insertion may re-use the memory locations of erased elements.
- 2 Element blocks which contain elements are referred to as *active blocks*, those which do not are referred to as *reserved blocks*. Active blocks which become empty of elements are either deallocated or become reserved blocks. Reserved blocks become active blocks when they are used to store elements. A user can create additional reserved blocks by calling `reserve`.
- 3 Erasures use unspecified techniques of constant time complexity to identify the memory locations of erased elements, which are subsequently skipped during iteration, as opposed to relocating subsequent elements during erasure.
- 4 Active block capacities have an implementation-defined growth factor (which need not be integral), for example a new active block's capacity could be equal to the summed capacities of the pre-existing active blocks.
- 5 Limits can be placed on both the minimum and maximum element capacities of element blocks, both by users and implementations.
 - (5.1) — The minimum limit shall be no larger than the maximum limit.
 - (5.2) —

When limits are not specified by a user during construction, the implementation's default limits are used.

- (5.3) — The default limits of an implementation are not guaranteed to be the same as the minimum and maximum possible capacities for an implementation's element blocks.

[*Note 1*: To allow latitude for both implementation-specific and user-directed optimization. — *end note*]

The latter are defined as hard limits. The maximum hard limit shall be no larger than `std::allocator_traits<Allocator>::max_size()`.

- (5.4) — If user-specified limits are not within hard limits, or if the specified minimum limit is greater than the specified maximum limit, the behavior is undefined.
- (5.5) — An element block is said to be *within the bounds* of a pair of minimum/maximum limits when its capacity is greater-or-equal-to the minimum limit and less-than-or-equal-to the maximum limit.

- 6 A hive conforms to the requirements for containers ([[container.reqmts](#)]), with the exception of operators `==` and `!=`. A hive also meets the requirements of a reversible container ([[container.rev.reqmts](#)]), of an allocator-aware container ([[container.alloc.reqmts](#)]), and some of the requirements of a sequence container ([[sequence.reqmts](#)]). Descriptions are provided here only for operations on hive that are not described in that table or for operations where there is additional semantic information.

- 7 The iterators of hive meet the *Cpp17BidirectionalIterator* requirements but also model `three_way_comparable<strong_ordering>`.

```
namespace std {
    template<class T, class Allocator = allocator<T>>
    class hive {
    public:
        // types
        using value_type = T;
        using allocator_type = Allocator;
        using pointer = allocator_traits<Allocator>::pointer;
        using const_pointer = allocator_traits<Allocator>::const_pointer;
        using reference = value_type&
        using const_reference = const value_type&
        using size_type = implementation-defined;
        // see [container.requirements]
        using difference_type = implementation-defined;
        // see [container.requirements]
        using iterator = implementation-defined;
        // see [container.requirements]
```

```

    using const_iterator = implementation-defined;
// see [container.requirements]
    using reverse_iterator = std::reverse_iterator<iterator>;
// see [container.requirements]
    using const_reverse_iterator = std::reverse_iterator<const_iterator>; // see [container.requirements]

    // [hive.cons], construct/copy/destroy
    constexpr hive() noexcept(noexcept(Allocator())) : hive(Allocator()) {}
    constexpr explicit hive(const Allocator&) noexcept;
    constexpr explicit hive(hive_limits block_limits) : hive(block_limits, Allocator()) {}
    constexpr hive(hive_limits block_limits, const Allocator&);
    constexpr explicit hive(size_type n, const Allocator& = Allocator());
    constexpr hive(size_type n, hive_limits block_limits, const Allocator& = Allocator());
    constexpr hive(size_type n, const T& value, const Allocator& = Allocator());
    constexpr hive(size_type n, const T& value, hive_limits block_limits, const Allocator& = Allocator());
    template<class InputIterator>
    constexpr hive(InputIterator first, InputIterator last, const Allocator& = Allocator());
    template<class InputIterator>
    constexpr hive(InputIterator first, InputIterator last, hive_limits block_limits,
        const Allocator& = Allocator());
    template<container-compatible-range<T> R>
    constexpr hive(from_range_t, R&& rg, const Allocator& = Allocator());
    template<container-compatible-range<T> R>
    constexpr hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& = Allocator());
    constexpr hive(const hive& x);
    constexpr hive(hive&&) noexcept;
    constexpr hive(const hive& x, const type_identity_t<Allocator>& alloc);
    constexpr hive(hive&&, const type_identity_t<Allocator>& alloc);
    constexpr hive(initializer_list<T> il, const Allocator& = Allocator());
    constexpr hive(initializer_list<T> il, hive_limits block_limits, const Allocator& = Allocator());
    constexpr ~hive();

```

```

constexpr hive& operator=(const hive& x);
constexpr hive& operator=(hive&& x) noexcept(see below);
constexpr hive& operator=(initializer_list<T>);
template<class InputIterator>
    constexpr void assign(InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr void assign_range(R&& rg);
constexpr void assign(size_type n, const T& t);
constexpr void assign(initializer_list<T>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator          begin() noexcept;
constexpr const_iterator    begin() const noexcept;
constexpr iterator          end() noexcept;
constexpr const_iterator    end() const noexcept;
constexpr reverse_iterator  rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator  rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;
constexpr const_iterator    cbegin() const noexcept;
constexpr const_iterator    cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// [hive.capacity], capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr size_type capacity() const noexcept;
constexpr void reserve(size_type n);
constexpr void shrink_to_fit();
constexpr void trim_capacity() noexcept;
constexpr void trim_capacity(size_type n) noexcept;
constexpr hive_limits block_capacity_limits() const noexcept;
static constexpr hive_limits block_capacity_default_limits() noexcept;
static constexpr hive_limits block_capacity_hard_limits() noexcept;
constexpr void reshape(hive_limits block_limits);

// [hive.modifiers], modifiers
template<class... Args> constexpr iterator emplace(Args&&... args);
template<class... Args> constexpr iterator emplace_hint(const_iterator hint, Args&&... args);

```

```

constexpr iterator insert(const T& x);
constexpr iterator insert(T&& x);
constexpr iterator insert(const_iterator hint, const T& x);
constexpr iterator insert(const_iterator hint, T&& x);
constexpr void insert(initializer_list<T> il);
template<container-compatible-range<T> R>
    constexpr void insert_range(R&& rg);
template<class InputIterator>
    constexpr void insert(InputIterator first, InputIterator la
st);
    constexpr void insert(size_type n, const T& x);

constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator
last);
constexpr void swap(hive&) noexcept(see below);
constexpr void clear() noexcept;

// [hive.operations], hive operations
constexpr void splice(hive& x);
constexpr void splice(hive&& x);
template<class BinaryPredicate = equal_to<T>>
    constexpr size_type unique(BinaryPredicate binary_pred = Bi
naryPredicate());

template<class Compare = less<T>>
    constexpr void sort(Compare comp = Compare());

iterator get_iterator(const_pointer p) noexcept;
const_iterator get_iterator(const_pointer p) const noexcept;

private:
    hive_limits current_limits = implementation-defined;    // expos
ition only
};

template<class InputIterator, class Allocator = allocator<iter-v
alue-type<InputIterator>>>
    hive(InputIterator, InputIterator, Allocator = Allocator())
        -> hive<iter-value-type<InputIterator>, Allocator>;

template<class InputIterator, class Allocator = allocator<iter-v
alue-type<InputIterator>>>
    hive(InputIterator, InputIterator, hive_limits, Allocator = A
lllocator())
        -> hive<iter-value-type<InputIterator>, Allocator>;

```

```

template<ranges::input_range R, class Allocator = allocator<rang
es::range_value_t<R>>>
    hive(from_range_t, R&&, Allocator = Allocator())
        -> hive<ranges::range_value_t<R>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<rang
es::range_value_t<R>>>
    hive(from_range_t, R&&, hive_limits, Allocator = Allocator())
        -> hive<ranges::range_value_t<R>, Allocator>;
}

```

23.3.9.2 Constructors, copy, and assignment

[hive.cons]

```
constexpr explicit hive(const Allocator&) noexcept;
```

1 *Effects:* Constructs an empty hive, using the specified allocator.

2 *Complexity:* Constant.

```
constexpr hive(hive_limits block_limits, const Allocator&);
```

3 *Effects:* Constructs an empty hive, using the specified allocator. Initializes *current-limits* with *block_limits*.

4 *Complexity:* Constant.

```
constexpr explicit hive(size_type n, const Allocator& = Allocator());
constexpr hive(size_type n, hive_limits block_limits, const Allocator&
= Allocator());
```

5 *Preconditions:* T is *Cpp17DefaultInsertable* into hive.

6 *Effects:* Constructs a hive with n default-inserted elements, using the specified allocator. If the second overload is called, also initializes *current-limits* with *block_limits*.

7 *Complexity:* Linear in n.

```
constexpr hive(size_type n, const T& value, const Allocator& =
Allocator());
constexpr hive(size_type n, const T& value, hive_limits block_limits,
const Allocator& = Allocator());
```

8 *Preconditions:* T is *Cpp17CopyInsertable* into hive.

9 *Effects:* Constructs a hive with n copies of value, using the specified allocator. If the second overload is called, also initializes *current-limits* with *block_*-*limits*.

10 *Complexity:* Linear in n.

```

template<class InputIterator>
    constexpr hive(InputIterator first, InputIterator last, const
Allocator& = Allocator());
template<class InputIterator>
    constexpr hive(InputIterator first, InputIterator last, hive_limits
block_limits,
        const Allocator& = Allocator());

```

11 *Effects:* Constructs a hive equal to the range [first, last), using the specified allocator. If the second overload is called, also initializes *current-limits* with block_limits.

12 *Complexity:* Linear in distance(first, last).

```

template<container-compatible-range<T> R>
    constexpr hive(from_range_t, R&& rg, const Allocator& = Allocator());
template<container-compatible-range<T> R>
    constexpr hive(from_range_t, R&& rg, hive_limits block_limits, const
Allocator& = Allocator());

```

13 *Effects:* Constructs a hive object with the elements of the range rg, using the specified allocator. If the second overload is called, also initializes *current-limits* with block_limits.

14 *Complexity:* Linear in ranges::distance(rg).

```

constexpr hive(const hive& x);
constexpr hive(const hive& x, const type_identity_t<Allocator>& alloc);

```

15 *Preconditions:* T is Cpp17CopyInsertable into hive.

16 *Effects:* Constructs a hive object with the elements of x. If the second overload is called, uses alloc. Initializes *current-limits* with x.current-limits.

17 *Complexity:* Linear in x.size().

```

constexpr hive(hive&& x) noexcept;
constexpr hive(hive&& x, const type_identity_t<Allocator>& alloc);

```

18 *Preconditions:* For the second overload, when allocator_traits<alloc>::is_always_equal::value is false, T meets the Cpp17MoveInsertable requirements.

19 *Effects:* When the first overload is called, or the second overload is called and alloc == x.get_allocator() is true, *current-limits* is set to x.current-limits and each element block is moved from x into *this. Pointers and references to the elements of x now refer to those same elements but as members of *this. Iterators referring to the elements of x will continue to refer to their elements, but they now behave as iterators into *this.

If the second overload is called and `alloc == x.get_allocator()` is `false`, each element in `x` is moved into `*this`. References, pointers and iterators referring to the elements of `x`, as well as the past-the-end iterator of `x`, are invalidated.

20 *Postconditions:* `x.empty()` is `true`.

21 *Complexity:* If the second overload is called and `alloc == x.get_allocator()` is `false`, linear in `x.size()`. Otherwise constant.

```
constexpr hive(initializer_list<T> il, const Allocator& = Allocator());
constexpr hive(initializer_list<T> il, hive_limits block_limits, const
Allocator& = Allocator());
```

22 *Preconditions:* `T` is *Cpp17CopyInsertable* into `hive`.

23 *Effects:* Constructs a `hive` object with the elements of `il`, using the specified allocator. If the second overload is called, also initializes *current-limits* with `block_limits`.

24 *Complexity:* Linear in `il.size()`.

```
constexpr hive& operator=(const hive& x);
```

25 *Preconditions:* `T` is *Cpp17CopyInsertable* into `hive` and *Cpp17CopyAssignable*.

26 *Effects:* All elements in `*this` are either copy-assigned to, or destroyed. All elements in `x` are copied into `*this`.

[Note 1: *current-limits* is unchanged. — end note]

27 *Complexity:* Linear in `size() + x.size()`.

```
constexpr hive& operator=(hive&& x)
noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
allocator_traits<Allocator>::is_always_equal::value);
```

28 *Preconditions:* When

```
(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
allocator_traits<Allocator>::is_always_equal::value)
```

is `false`, `T` is *Cpp17MoveInsertable* into `hive` and *Cpp17MoveAssignable*.

29 *Effects:* Each element in `*this` is either move-assigned to, or destroyed. When

```
(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
get_allocator() == x.get_allocator())
```

is `true`, *current-limits* is set to `x.current-limits` and each element block is moved from `x` into `*this`. Pointers and references to the elements of `x` now

refer to those same elements but as members of `*this`. Iterators referring to the elements of `x` will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

When

```
(allocator_traits<Allocator>::propagate_on_container_move_as  
signment::value ||  
get_allocator() == x.get_allocator())
```

is `false`, each element in `x` is moved into `*this`. References, pointers and iterators referring to the elements of `x`, as well as the past-the-end iterator of `x`, are invalidated.

30 *Postconditions:* `x.empty()` is `true`.

31 *Complexity:* Linear in `size()`. If

```
(allocator_traits<Allocator>::propagate_on_container_move_as  
signment::value ||  
get_allocator() == x.get_allocator())
```

is `false`, also linear in `x.size()`.

23.3.9.3 Capacity

[hive.capacity]

```
constexpr size_type capacity() const noexcept;
```

1 *Returns:* The total number of elements that `*this` can hold without requiring allocation of more element blocks.

2 *Complexity:* Constant.

```
constexpr void reserve(size_type n);
```

3 *Effects:* If `n <= capacity()` is `true`, there are no effects. Otherwise increases `capacity()` by allocating reserved blocks.

4 *Postconditions:* `capacity() >= n` is `true`.

5 *Throws:* `length_error` if `n > max_size()`, as well as any exceptions thrown by the allocator.

6 *Complexity:* Linear in the number of reserved blocks allocated.

7 *Remarks:* The size of the sequence is not changed. All references, pointers, and iterators referring to elements in `*this`, as well as the past-the-end iterator, remain valid.

```
constexpr void shrink_to_fit();
```

8 *Preconditions:* `T` is `Cpp17MoveInsertable` into `hive`.

9

Effects: `shrink_to_fit` is a non-binding request to reduce `capacity()` to be closer to `size()`.

[*Note 1:* The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*]

It does not increase `capacity()`, but may reduce `capacity()`. It may reallocate elements. If `capacity()` is already equal to `size()`, there are no effects. If an exception is thrown during allocation of a new element block, `capacity()` may be reduced and reallocation may occur. Otherwise if an exception is thrown, the effects are unspecified.

10 *Complexity:* If reallocation happens, linear in the size of the sequence.

11 *Remarks:* If reallocation happens, the order of the elements in `*this` may change and all references, pointers, and iterators referring to the elements in `*this`, as well as the past-the-end iterator, are invalidated.

```
constexpr void trim_capacity() noexcept;
constexpr void trim_capacity(size_type n) noexcept;
```

12 *Effects:* For the first overload, all reserved blocks are deallocated, and `capacity()` is reduced accordingly. For the second overload, `capacity()` is reduced to no less than `n`.

13 *Complexity:* Linear in the number of reserved blocks deallocated.

14 *Remarks:* All references, pointers, and iterators referring to elements in `*this`, as well as the past-the-end iterator, remain valid.

```
constexpr hive_limits block_capacity_limits() const noexcept;
```

15 *Returns:* *current-limits*.

16 *Complexity:* Constant.

```
static constexpr hive_limits block_capacity_default_limits() noexcept;
```

17 *Returns:* A `hive_limits` struct with the `min` and `max` members set to the implementation's default limits.

18 *Complexity:* Constant.

```
static constexpr hive_limits block_capacity_hard_limits() noexcept;
```

19 *Returns:* A `hive_limits` struct with the `min` and `max` members set to the implementation's hard limits.

20 *Complexity:* Constant.

```
constexpr void reshape(hive_limits block_limits);
```

Preconditions: T is *Cpp17MoveInsertable* into hive.

22 *Effects:* For any active blocks not within the bounds of `block_limits`, the elements within those active blocks are reallocated to new or existing element blocks which are within the bounds. Any element blocks not within the bounds of `block_limits` are deallocated. If an exception is thrown during allocation of a new element block, `capacity()` may be reduced, reallocation may occur, and *current-limits* may be assigned a value other than `block_limits`. Otherwise `block_limits` is assigned to *current-limits*. If any other exception is thrown the effects are unspecified.

23 *Postconditions:* `size()` is unchanged.

24 *Complexity:* Linear in the number of element blocks in `*this`. If reallocation happens, also linear in the number of elements reallocated.

25 *Remarks:* This operation may change `capacity()`. If reallocation happens, the order of the elements in `*this` may change. Reallocation invalidates all references, pointers, and iterators referring to the elements in `*this`, as well as the past-the-end iterator.

[Note 2: If no reallocation happens, they remain valid. — end note]

23.3.9.4 Modifiers

[hive.modifiers]

```
template<class... Args> constexpr iterator emplace(Args&&... args);  
template<class... Args> constexpr iterator emplace_hint(const_iterator  
hint, Args&&... args);
```

1 *Preconditions:* T is *Cpp17EmplaceConstructible* into hive from args.

2 *Effects:* Inserts an object of type T constructed with `std::forward<Args>(args)...`. The hint parameter is ignored. If an exception is thrown, there are no effects.

[Note 1: args can directly or indirectly refer to a value in `*this`. — end note]

3 *Returns:* An iterator that points to the new element.

4 *Complexity:* Constant. Exactly one object of type T is constructed.

5 *Remarks:* Invalidates the past-the-end iterator.

```
constexpr iterator insert(const T& x);  
constexpr iterator insert(const_iterator hint, const T& x);  
constexpr iterator insert(T&& x);  
constexpr iterator insert(const_iterator hint, T&& x);
```

6 *Effects:* Equivalent to: `return emplace(std::forward<decltype(x)>(x));`

[Note 2: The hint parameter is ignored. — end note]

```
constexpr void insert(initializer_list<T> rg);
```

```
template<container-compatible-range<T> R>
```

```
constexpr void insert_range(R&& rg);
```

7 *Preconditions:* T is *Cpp17EmplaceInsertable* into hive from **ranges::begin(rg)*. *rg* and **this* do not overlap.

8 *Effects:* Inserts copies of elements in *rg*. Each iterator in the range *rg* is dereferenced exactly once.

9 *Complexity:* Linear in the number of elements inserted. Exactly one object of type T is constructed for each element inserted.

10 *Remarks:* If an element is inserted, invalidates the past-the-end iterator.

```
constexpr void insert(size_type n, const T& x);
```

11 *Preconditions:* T is *Cpp17CopyInsertable* into hive.

12 *Effects:* Inserts *n* copies of *x*.

13 *Complexity:* Linear in *n*. Exactly one object of type T is constructed for each element inserted.

14 *Remarks:* If an element is inserted, invalidates the past-the-end iterator.

```
template<class InputIterator>
```

```
constexpr void insert(InputIterator first, InputIterator last);
```

15 *Effects:* Equivalent to *insert_range(ranges::subrange(first, last))*.

```
constexpr iterator erase(const_iterator position);
```

```
constexpr iterator erase(const_iterator first, const_iterator last);
```

16 *Complexity:* Linear in the number of elements erased. Additionally, if any active blocks become empty of elements as a result of the function call, at worst linear in the number of element blocks.

17 *Remarks:* Invalidates references, pointers and iterators referring to the erased elements. An erase operation that erases the last element in **this* also invalidates the past-the-end iterator.

```
constexpr void swap(hive& x)
```

```
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
```

```
allocator_traits<Allocator>::is_always_equal::value);
```

18 *Effects:* Exchanges the contents, *capacity()*, and *current-limits* of **this* with that of *x*.

19 *Complexity:* Constant.

- ¹ In this subclause, arguments for a template parameter named `Predicate` or `BinaryPredicate` shall meet the corresponding requirements in [\[algorithms.requirements\]](#). The semantics of `i + n` and `i - n`, where `i` is an iterator into the hive and `n` is an integer, are the same as those of `next(i, n)` and `prev(i, n)`, respectively. For sort, the definitions and requirements in [\[alg.sorting\]](#) apply.

```
constexpr void splice(hive& x);
constexpr void splice(hive&& x);
```

- ² *Preconditions:* `get_allocator() == x.get_allocator()` is `true`.
- ³ *Effects:* If `addressof(x) == this` is `true`, the behavior is erroneous and there are no effects. Otherwise, inserts the contents of `x` into `*this` and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.
- ⁴ *Throws:* `length_error` if any of `x`'s active blocks are not within the bounds of *current-limits*.
- ⁵ *Complexity:* Linear in the sum of all element blocks in `x` plus all element blocks in `*this`.
- ⁶ *Remarks:* Reserved blocks in `x` are not transferred into `*this`. If `addressof(x) == this` is `false`, invalidates the past-the-end iterator for both `x` and `*this`.

```
template<class BinaryPredicate = equal_to<T>>
constexpr size_type unique(BinaryPredicate binary_pred =
BinaryPredicate());
```

- ⁷ *Preconditions:* `binary_pred` is an equivalence relation.
- ⁸ *Effects:* Erases all but the first element from every consecutive group of equivalent elements. That is, for a nonempty hive, erases all elements referred to by the iterator `i` in the range `[begin() + 1, end())` for which `binary_pred(*i, *(i - 1))` is `true`.
- ⁹ *Returns:* The number of elements erased.
- ¹⁰ *Throws:* Nothing unless an exception is thrown by the predicate.
- ¹¹ *Complexity:* If `empty()` is `false`, exactly `size() - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

Remarks: Invalidates references, pointers, and iterators referring to the erased elements. If the last element in `*this` is erased, also invalidates the past-the-end iterator.

```
template<class Compare = less<T>>
```

```
constexpr void sort(Compare comp = Compare());
```

13 *Preconditions:* `T` is *Cpp17MoveInsertable* into *hive*, *Cpp17MoveAssignable*, and *Cpp17Swappable*.

14 *Effects:* Sorts `*this` according to the `comp` function object. If an exception is thrown, the order of the elements in `*this` is unspecified.

15 *Complexity:* $\mathcal{O}(N \log N)$ comparisons, where N is `size()`.

16 *Remarks:* May allocate. References, pointers, and iterators referring to elements in `*this`, as well as the past-the-end iterator, may be invalidated.

[Note 1: Not required to be stable[[algorithm.stable](#)]. — end note]

```
iterator get_iterator(const_pointer p) noexcept;
```

```
const_iterator get_iterator(const_pointer p) const noexcept;
```

17 *Preconditions:* `p` points to an element in `*this`.

18 *Returns:* An iterator or `const_iterator` pointing to the same element as `p`.

19 *Complexity:* Linear in the number of active blocks in `*this`.

23.3.9.6 Erasure

[hive.erasure]

```
template<class T, class Allocator, class U = T>
```

```
constexpr typename hive<T, Allocator>::size_type  
erase(hive<T, Allocator>& c, const U& value);
```

1 *Effects:* Equivalent to:

```
return erase_if(c, [&](const auto& elem) -> bool { return elem  
== value; });
```

```
template<class T, class Allocator, class Predicate>
```

```
constexpr typename hive<T, Allocator>::size_type  
erase_if(hive<T, Allocator>& c, Predicate pred);
```

2 *Effects:* Equivalent to:

```
auto original_size = c.size();  
for (auto i = c.begin(), last = c.end(); i != last; ) {  
    if (pred(*i)) {  
        i = c.erase(i);  
    } else {  
        ++i;  
    }  
}
```

```
}  
return original_size - c.size();
```

Feature test macro

17.3.2 Header <version> synopsis [version.syn]

```
#define __cpp_lib_constexpr_hive 2026??L // also in <hive>
```